server2 infrastructure preparation (S03, S15b-4c cache)

Status: reproducible local test environment and staging plan. S03 changed no AWS resources or routing; the account root provisioned the staging bucket and runtime role on 2026-10-15 (S10f, see the worksheet below). S01/S04 are integrated. The foundation binary serves health and loopback metrics, without weather/S3/provider runtime routes. Architecture and S03 contract describe the boundaries.

Start and verify locally

Prerequisites: Python 3.11+, the pinned Rust 1.99 toolchain, and a built release binary. Run from `server2/`. Docker, MongoDB, SQLite, provider keys and AWS credentials are not required. Existing provider fixture bytes were copied once with provenance and hashes under `config/fixtures/`; the runtime never reads legacy files. These are repository oracle inputs, not new live measurements.

```
cargo build --locked --release
python3 deploy/local/test_local.py
python3 deploy/local/smoke.py --binary target/release/server2
```

The distinct smoke starts the foundation and two peers on OS-selected loopback ports, reads a recorded KMA response, gzip-encodes its exact bytes, tests conditional PUT/412 and HEAD/GET metadata, injects committed PUT response failure/loss, and reconciles the stored object. It then sends SIGTERM and verifies owned child/PIDs, all listeners and the ready file are gone. Unknown endpoints or credentials are rejected locally. No peer ever forwards a request to AWS or providers.

For an interactive session, create a private `server2-s03-*` directory in the system temporary directory and keep the stack in the foreground:

```
S03_DIR=$(mktemp -d "${TMPDIR:-/tmp}/server2-s03-XXXXXXXX")
python3 deploy/local/stack.py --binary target/release/server2 --ready "$S03_DIR/ready.json"
```

Read `ready.json` from a second terminal for the local endpoints and dummy keys. It becomes visible only after a complete JSON file is written and flushed in the same private directory, then atomically hard-linked with no replacement. The first open must parse complete JSON; do not retry JSON decode failures to hide a race. An existing file, symlink or dangling symlink is preserved and startup fails. Unsupported hard links fail closed; there is no overwrite/replace fallback. Ordinary write/publication failure cleans owned temporary files and processes. Normal stop removes the ready file only if its regular-file device/inode still matches this launch; a replacement entry is preserved.

The additional release-launcher observation check opens readiness immediately when available for 12 independent starts, without decode retries, and verifies health plus owned process, port, ready and temporary-file cleanup for each start:

```
python3 deploy/local/smoke.py --binary target/release/server2 --ready-observations 12
```

Ctrl-C/SIGTERM stops only the owned foundation child and peers. Remove the now empty task directory after stop. Ready paths must be absent, under a private task temporary directory; the launcher refuses external/non-owned binaries. Bind/startup failure exits nonzero and cleans owned children; stop does not contact production. Never use this test stack as a public server or point it at production credentials.

SIGKILL or a hard launcher crash bypasses cleanup: the Rust child may survive and ready.json or a private `.ready-*` temporary file may remain. A ready file is not proof of a live/current launcher. Before recovery, record its launcher/foundation PIDs and endpoints. Use a process listing (for example `ps -p <foundation_pid> -o pid,ppid,lstart,command`) to confirm the child is the exact owned release binary from this run, with the expected start time; never kill a PID solely because it appears in a stale ready file. Send SIGTERM only to that confirmed child, wait for exit, and confirm all recorded loopback listeners are closed. If identity cannot be established, stop and inspect manually. Only after confirming this run is stopped, remove its own stale ready file and this run's private temporary files/directory, then start with a fresh absent ready path. The launcher implements normal-signal cleanup, not parent-death supervision.

Local S3 subset

Path-style `/server2-local/key` supports PUT/HEAD/GET; bucket GET with `list-type=2` provides lexical prefix pagination. PUT requires `Content-MD5` of compressed bytes and `If-None-Match: *` or `If-Match: "etag"`. MD5 failure is 400; an existing raw key or failed CAS is 412. There is no deletion. HEAD/GET preserve exact compressed body length, Content-Type, ETag, version ID and `x-amz-meta-*`; an explicit `versionId` retrieves that retained local revision. Range, multipart and version listing are unsupported and rejected, rather than simulated. LIST accepts only list-type, prefix, max-keys and continuation-token (plus dummy signing fields); delimiter, start-after, fetch-owner, encoding-type and unknown options return 501. Duplicate query fields return 400. No Content-Encoding is added. The fixture provider accepts only recorded GET/HEAD paths.

The test header `X-Server2-Test-Key: server2-local`, or a presigned-query shape with dummy access key `server2-local` and a 64-hex signature, identifies a request. Dummy secret `server2-local-secret` and region `us-east-1` are solely for SDK tests. The peer does not verify SigV4 signatures, IAM, TLS, clock/skew or AWS retries. Its finite volatile version map vanishes at process exit; it is not a server2 serving store, persistent emulator, MinIO replacement or AWS latency benchmark. Real AWS signing/conditional headers and role restrictions require S09 verification.

Default bounds: 1 MiB per body, 32 MiB accounted body/metadata/key/version overhead, 2 KiB user metadata (original UTF-8 wire key/value bytes, excluding the wire prefix), 8 KiB original request-header key/value bytes, 4,096 object keys, 64 versions/key and 16 handler connections. Both the per-operation socket timeout and absolute admitted-connection lifetime are 2 seconds; dripping header/body bytes cannot extend that lifetime. These bound local test growth; they are not an exact RSS promise or a claim to support every S3 object size. S05 production body caps are separate. The S3 metadata rules define the real service limits.

Authenticated test-only POST `/__test/fault/error-after-put?key=<encoded-key>` or `/__test/fault/drop-after-put?key=<encoded-key>` affects the next successful PUT for that key. Both commit first; the first returns 500, the second drops the response. Reconcile with HEAD/GET rather than assuming no object was stored. No fault endpoint exists in the production runtime.

Staging approval worksheet

The prerequisites JSON records the resources the account root provisioned on 2026-10-15 at AK's decision (S10f, #2759): account `141248341265`, region `ap-northeast-2`, bucket `tw-server2-141248341265-apne2` and the role and instance profile `tw-server2-runtime`, not yet attached to an instance. Owners and the monthly ceiling remain unset. S03 does not contain an apply/provision command. Resource approval is separate from this PR or local start. Do not reuse the local dummy keys outside tests.

- Infrastructure owner confirms the origin region/account and private bucket in that same region, versioning Enabled, all Block Public Access settings, BucketOwnerEnforced ownership and default SSE-S3 encryption. No lifecycle deletion or transition is selected now. S09 measures version accumulation and retrieval costs; no Flexible/Deep Archive on synchronous serving paths. - Runtime role gets only prefix-scoped ListBucket, GetObject and PutObject; historical index/state versions additionally use GetObjectVersion and scoped ListBucketVersions. HEAD uses GetObject authorization. There are no DeleteObject, delete-version, bucket-admin, EC2-write or broad AWS permissions in the role template. The provisioned inline policy `tw-server2-runtime-s3` equals this template with the bucket name substituted, including `geocode/address/v1/*` (read back on 2026-10-15); the provisioned bucket policy equals the bucket template. AK's IAM policy simulation allowed the server2 prefixes and denied delete, other prefixes, other buckets and bucket administration. Encryption with customer KMS keys would require a new least-privilege review; it is not silently covered by this template. Unverified IAM risk: the ListBucket grant has an s3:prefix condition. AWS documents that missing-key reads return 404 when ListBucket is permitted, otherwise 403; whether this conditional grant suffices for HEAD/GET is not established by local tests. S09 must use the actual runtime role to HEAD and GET a known-absent raw key and catalog key and require 404/NoSuchKey, while separately proving that unauthorized reads return 403. The `s3_readiness` example (`cargo run --locked --release --example s3_readiness -- LABEL` from `server2/`) performs these reads, the conditional writes and, with the instance role, the 403 check outside the server2 prefixes. Its 2026-10-15 run used the operator IAM user, not the role (sanitized evidence); the runtime-role run remains for the EC2 deployment. Record sanitized status, role/policy revision, region and time; a mismatch blocks readiness and requires reviewed policy/error-classification correction. Do not treat 403 as absence or widen permissions from this plan. See the GetObject missing-key rule. - The bucket template rejects non-TLS and unconditional raw writes, and requires conditional catalog/ state publication. AWS supports conditional-write policy enforcement. The runtime role does not change versioning/policies. Operator approval covers installation and a deny-case test with the actual role, including attempted delete. - Provider/quota owner: since S10e (#2753, AK 2026-10-15) server2 uses the legacy keys under their legacy names, from `server2/.env`, a private copy of the legacy `server/.env`: `DONGNAE_SECRET_KEYS`, `AIRKOREA_SECRET_KEYS`, `VC_SECRET_KEY`, `WAQI_SECRET_KEY`, `OWM_SECRET_KEY`, `GOOGLE_SECRET_KEY`, `GEOCODER_KAKAO_KEYS`, `GEOCODER_GOOGLE_KEY`; the server2-only `SERVER2_S3_BUCKET` and `SERVER2_S3_REGION` are added there. S3 credentials come from the EC2 instance role through IMDSv2 while

AWS_ACCESS_KEY/AWS_SECRET_KEY are unset or the legacy placeholder (the operator's file holds the placeholder); the legacy TWA_S3_REGION is not read. Server2 and legacy then share each key's provider quota. Values stay outside Git/logs. Confirm daily and peak calls/records for warm-up, cold geocoding, hourly/2-minute demand capture and required legacy collectors. Count data.go.kr calls and VC provider records, not logical requests; keep remaining legacy collection quota/cost visible. - Operations owner confirms network ingress from the existing origin routing, loopback-only metrics, provider egress allowlist and a same-region TLS S3 path. Evaluate an S3 gateway endpoint and network charges with the actual topology. No gateway/Lambda/CloudFront changes are performed by S03.

Host, readiness and Spot interruption

Start with one Spot host and one Tokio multithreaded API process. ARM c7g.large or x86 `c6i.large` are measurement candidates, each 2 vCPU/4 GiB; there is no measured host choice or price claim in this PR. Availability, current Spot price and budget must be checked in the approved region before provisioning. Increase memory/CPU when S09 finds pressure; do not introduce multiple cache-owning API processes first.

S09 must build Rust 1.99 on the actual Linux target (`aarch64-unknown-linux-musl` or `x86_64-unknown-linux-musl`), verify release startup/signals/TLS CA/DNS and SDK behavior, and report RSS/CPU, cold p50/p95/p99, S3 throughput and provider budget headroom. Leave a proposed 25% host headroom until real workload sizing changes it. S15b-4c adds a disposable local raw cache to the root disk; S3 remains its source of truth. No local weather database, SQLite, Redis or shared-instance cache is introduced. The volume is replaceable: even an EBS root that survives an instance restart is never treated as durable application state.

`/health` proves process liveness only. Before a route receives traffic, later phases must prove configuration/keys, S3 role access, provider reservations, family readiness and bounded cold warm-up. The foundation does not yet implement that readiness gate or an IMDS watcher.

For a separately approved production drain, an IMDSv2 interruption/rebalance watcher marks the instance unready, removes new routing, stops new provider budget reservations/capture work, bounds completion of owned raw/fetch-group publications, then sends SIGTERM. Persisted-before-response acknowledgment is the durability boundary; the interruption signal is not. Spot notices are best effort, and hibernation does not provide the usual two-minute window. A rebalance recommendation may arrive alongside the interruption notice. Unexpected-loss testing therefore remains required even with a watcher. Local SIGTERM smoke is not an actual Spot test.

Rollback and later scale-out

During coexistence, use the existing per-version family switches and rehearse CloudFront-aware rollback and combined provider quotas. AK's later retirement instruction supersedes the original S03 cold-cell forwarding plan: after server2 go-live, native requests must not depend on a running legacy server or precollection. Missing inputs use request-time acquisition and legacy partial missing-data/error shapes. S20 must verify cold cells, loss/restart, parity and budgets before retirement; this document is not a completed cutover claim.

Automatic failed/unknown push resend remains prohibited. AK later authorized a one-time read-only copy of push registrations (S18a), superseding the original no-migration decision. Preserve accepted S3 registrations during rollback; do not broadcast on startup. Weather/provider retries remain their separate contract.

Future multiple instances keep independent memory caches and share S3. Before that rollout, implement durable provider admission, fetch/background ownership, CAS contention, readiness/drain and bounded warm-up. This is a plan only; no shared cache or multi-instance deployment is introduced.

Disposable local cache and required disk capacity (S15b-4c)

Provision at least 32 GiB gp3 root capacity for the initial single-process configuration below. This is a capacity recommendation with explicit assumptions, not a measured EC2 latency/IOPS or production demand result. Start with gp3's baseline 3,000 IOPS and 125 MiB/s; change performance provision only after real host measurements identify a need.

server2 does not automatically create an application disk cache unless a directory is configured. In the private `server2/.env` (copied from legacy, with provider names unchanged), add these real operational settings for this deployment:

```
SERVER2_LOCAL_CACHE_DIR=/var/cache/server2/raw
SERVER2_LOCAL_CACHE_MAX_BYTES=8589934592
```

Prepare a dedicated directory owned by the server2 service user with mode 0700; new cache files are 0600. A missing directory is created lazily, without any provider/S3 operation at settings parse time. A cap without a directory, malformed cap, or cap outside 4 KiB–1 TiB is a startup configuration error. A configured directory without a cap uses 8 GiB. Unset both settings to keep cache `None`. These settings do not select a provider or a validation mode. No private `.env` is read by the tests or measurement tool.

The cache holds verified immutable raw objects for `data-go-kr`, `kma-web`, `vc` and the exact `airkorea-image` / `airkorea-forecast-image` pair. AirKorea's image raws are included; decoded GIFs and packs are not added in this increment. Mutable catalogs, absence proofs, state, budgets, push tokens, geocode and exact request coordinates are excluded. Catalog/group publication checks still use S3. Filename and namespace hashes do not replace the raw codec's identity validation.

A root capacity lock serializes disk admission through temporary write, rename and cleanup. The byte cap covers managed raw plus temporary allocated file bytes across all hashed namespaces in this dedicated directory, including old bucket/region namespaces. Only the current namespace is evicted. If old data uses the remaining cap, a new fill is skipped and the request uses S3 normally; other namespaces are never reused or deleted. A lower cap does not promise to delete old namespaces that were already larger; it prevents new growth until space is available. Keep the same cap for all processes sharing a root; the supported initial deployment has one API process. A namespace owner lock prevents two cache managers from modifying the same namespace. Two admitted local operations bound copies/blocking work; lock contention bypasses optional cache work.

LRU eviction uses access order while running and file modification order after a restart. There are at most 131,072 entries per namespace. Symlinks are refused. Truncated/checksum-invalid/codec-invalid local files are discarded and fetched from S3. Cache loss, read errors, ownership conflicts, full disk and failed writes cannot acknowledge publication or turn a bad S3 object into valid data. Files left by an interrupted write are cleaned only in the owned namespace. A live cache directory deleted externally may remain bypassed until process restart; canonical reads remain correct. Do not put unrelated operator files in the cache directory or count it as the only copy of any state.

Size the root with this reusable formula (GiB, round up):

```
root >= (OS + retained binaries + bounded logs + global cache cap
         + peak temporary work outside that cap + swap) / (1 - free fraction)
initial = (8 + 1 + 2 + 8 + 2 + 0) / (1 - 0.25) = 28 GiB
recommended provision = 32 GiB gp3
```

OS 8 GiB, retained binaries 1 GiB, bounded sanitized logs 2 GiB and non-cache update/temporary work 2 GiB are operator budgets, not measured production usage. Swap is assumed off; add its size if configured. The cache cap already includes cache temporary files: do not add an extra 8 GiB or image copy for them. Directory/inode/lock overhead and filesystem variation fit in the 25% reserve; monitor `df -h`, `df -i` and allocated `du` on the entire cache root. Bound log rotation and binary retention to the listed budgets. Resize the root or lower the cap before headroom falls below 25%; never enlarge the cap without updating this calculation. With a 16 GiB cache cap the same formula needs at least 39 GiB (round to 40 GiB or more). More active regions increase misses/eviction, not the configured disk cap. Nationwide images are shared records and must not be multiplied by the number of active grids.

The selected local measurement (../evidence/tasks/issue-2742-s15b4c/cache-measurement.json) uses an incompressible synthetic 6 MiB raw payload, not a real GIF decode or AWS benchmark. Its gzip wire is about 6 MiB and its cache allocation about 6 MiB; exact bytes, temporary reservation peak, binary sizes and execution timestamp are in the receipt. Native HTTP with three seeded current publications returned the same response with cache unset, cold disk and empty-RAM/warm disk. Warm disk reduced raw GETs from 3 to 1, while 371 catalog GETs remained. Optional fills may be skipped under bounded local contention, so warm disk is not a promise of zero raw GETs. Local raw warm latency was slower in this sample; no cold p95 achievement is inferred. Real S3, EBS, full-data cold cells and GIF decoding must be measured in S19/S20. Use that evidence to decide whether decoded caching or O01 packs are necessary; do not assume a pack archive or precollection is required.

Run the reproducible local check with task-built debug/release examples:

```
python3 server2/tools/kma_cache_measure.py \
  --measure-binary /path/to/examples/kma_cache_measure \
  --native-binary /path/to/examples/loopback_server \
  --output reports/cache-measurement.json
```

It uses owned loopback S3/provider doubles and explicit fake credentials, never production `.env`, AWS or a real provider. The capture (../evidence/tasks/issue-2742-s15b4c/cache-measurement.png) and verification record (../evidence/tasks/issue-2742-s15b4c/verification.md) distinguish this functional observation from deployment readiness.

Evidence and limitations

The selected CLI capture and manifest bind the actual test output and release binary. Editable source is this document; the PDF manual includes the same operational boundaries. Unit tests cover wire boundaries separately from integrated process smoke. The local smoke performs no live AWS access, provider request, traffic replay, IAM application, musl Linux run, real Spot interruption, weather parity or latency/cost measurement. The separately authorized S10f bucket check with the operator IAM user is recorded in its sanitized evidence; the instance-role path itself has run only against a loopback IMDS double.

Actual local cache measurement

```
S15b-4c ACTUAL OWNED-LOOPBACK DEBUG HTTP
UTC 2026-10-16T07:33:51.903414+00:00
Partial KMA inputs; no AWS/provider/GIF decode or production p95 claim

cache-unset-empty-ram
  status=200  response=27582 B  elapsed=396.09 ms
  raw GET=3  catalog GET=371  legacy hops=0
cold-empty-ram-and-disk
  status=200  response=27582 B  elapsed=377.33 ms
  raw GET=3  catalog GET=371  legacy hops=0
warm-disk-empty-ram
  status=200  response=27582 B  elapsed=373.45 ms
  raw GET=1  catalog GET=371  legacy hops=0

SYNTHETIC IMAGE-SIZED RAW (not a valid GIF)
raw=6291456 B  gzip=6292469 B
cold-empty-ram-and-disk: 317.53 ms, hits=0, corrupt=0, allocated=6295552 B
warm-disk-empty-ram: 475.90 ms, hits=1, corrupt=0, allocated=6295552 B
corrupt-disk: 318.83 ms, hits=0, corrupt=1, allocated=6295552 B
missing-disk: 319.20 ms, hits=0, corrupt=0, allocated=6295552 B
```